

NumPy 线性代数讲义

Linear Algebra with NumPy

清华大学统计与数据科学系 | 2026.3.25

第一章 什么是 NumPy?

1.1 NumPy 简介

NumPy (**N**umerical **P**ython) 是 Python 中数值计算的核心库，几乎所有数据科学工作的基础都建立在它之上。

- 高效存储与计算**: 针对大型多维数组进行了深度优化
- 多维数组**: 支持任意维度的 ndarray 对象
- 丰富的数学函数**: 内置大量数学、统计、线性代数函数
- 随机数生成**: 覆盖几乎所有常见概率分布
- 生态基础**: pandas、SciPy、scikit-learn 均以 NumPy 为底层

1.2 为什么需要 NumPy?

数据科学 = 大规模数据上的数学运算。Python 原生列表虽然可以工作，但在大规模数据面前性能极差，可读性也容易变差。

纯 Python vs NumPy 对比

```
# 纯 Python 列表
my_list = list(range(1_000_000))
my_list2 = [x * 2 for x in my_list]  # 慢: 约 27.1 ms

# NumPy 向量化
import numpy as np
my_arr = np.arange(1_000_000)
my_arr2 = my_arr * 2  # 快: 约 1.04 ms (快约 26 倍!)
```

NumPy 速度优势的原因:

- 编译 C 代码**: 底层运算在编译好的 C 代码中执行，而非逐行 Python 解释
- Intel MKL / OpenBLAS**: 硬件级别高度优化的数学库
- SIMD 数据并行**: 一条 CPU 指令同时处理多个数据
- 连续内存布局**: 数组数据在内存中连续存储，大幅减少 CPU 缓存缺失（访问 CPU 缓存仅需 10ns, RAM 需 100ns）

核心理念

NumPy 的编程风格是「写公式而不是写算法」——代码看起来像数学，消除了显式循环，既可读又高效。

第二章 ndarray 对象

2.1 什么是 ndarray?

ndarray (N-dimensional array) 是 NumPy 的核心数据结构，具有以下特点：

- 同质固定类型：**数组中所有元素必须是相同的数据类型（如所有都是 float64）
- 任意维度：**支持 1D 向量、2D 矩阵，乃至更高维张量
- 快速逐元素运算：**所有运算均为向量化，无需手写循环

2.2 创建数组

从数据创建

```
arr1 = np.array([6, 7.5, 8, 0, 1])          # 1D 数组
arr2 = np.array([[1, 2, 3, 4], [5, 6, 7, 8]]) # 2D 数组（矩阵）
```

快速生成常用数组

函数	说明
np.zeros(10)	生成长度为 10 的全 0 数组
np.ones((3, 6))	生成 3×6 的全 1 矩阵
np.arange(15)	类似 range(15)，返回数组 [0..14]
np.arange(0, 5, 0.5)	步长为 0.5 的序列
np.eye(3)	3×3 单位矩阵（主对角线为 1）
np.repeat([1,2,3], 3)	每个元素重复 3 次: [1,1,1,2,2,2,3,3,3]
np.tile([1,2,3], 3)	整体重复 3 次: [1,2,3,1,2,3,1,2,3]
np.zeros_like(A)	与 A 形状相同的全 0 数组

2.3 数组属性

属性	含义
arr.ndim	数组的维度数 (1D→1, 2D→2, ...)
arr.shape	形状元组, 如 (5,) 或 (3,4)
arr.dtype	元素的数据类型, 如 float64、int64
arr.size	元素总数 = shape 各维度之积

2.4 数据类型 (dtype)

- **int64**: 64 位整数，默认整数类型
- **float64**: 64 位浮点数，默认浮点类型
- **bool**: 布尔值 (True/False)，常用于掩码过滤

同质性规则: 数组中若混有整数和浮点数，会自动向上转换为 float64:

```
np.array([1, 2, 3.5]).dtype    # float64
```

2.5 重塑 (reshape)

reshape() 可以在不复制数据的前提下改变数组形状，仅修改描述形状的元数据:

```
np.arange(12).reshape(4, 3)    # 变为 4×3 矩阵
np.arange(4).reshape(-1, 1)    # -1 表示自动推断，结果为 4×1 列向量
```

⚠ 注意

NumPy 默认按「行优先 (C order)」存储数据，即数据在内存中按行连续存放。
reshape 通常无需复制数据，非常高效。

2.6 数据集表示惯例

矩阵 X 的标准含义:

- **行 (axis=0) = 样本**: 每行是一个观测/用户/数据点
- **列 (axis=1) = 特征**: 每列是一个变量/特征

```
X = np.random.randn(100, 3)    # 100 个样本, 3 个特征
```

2.7 轴 (Axis) 的理解

很多聚合函数有 **axis** 参数，含义是「沿哪个方向计算」:

```
X.mean(axis=0)    # 沿行方向压缩 → 得到每列 (每个特征) 的均值, 形状 (3,)
X.mean(axis=1)    # 沿列方向压缩 → 得到每行 (每个样本) 的均值, 形状 (100,)
```

⚠ 记忆口诀

axis=0 沿「下」压 (消灭行, 留列), **axis=1** 沿「右」压 (消灭列, 留行)。

第三章 随机数生成

3.1 最常用的分布

```
# 连续均匀分布 [low, high)
np.random.uniform(0, 1, 100)

np.random.rand(3, 4)          # [0,1) 均匀, 旧接口简写

# 离散均匀分布 (整数)
```

```

np.random.randint(0, 10, 100)

# 正态分布
np.random.normal(loc=0, scale=1, size=100) # 均值 loc, 标准差 scale
np.random.randn(100)                       # 标准正态 N(0,1)

# 类别分布 (按概率随机选类)
np.random.choice(a=cats, p=probs, size=100) # a=类别列表, p=概率

```

3.2 新旧接口对比

旧接口 (legacy) : `np.random.xxx()`, 最常见, pandas/SciPy 也受其 seed 影响

新接口 (推荐) : `np.random.default_rng()`, 更快 (约 1.5-2x)、更「随机」、支持多个独立生成器实例

```

# 旧接口
np.random.seed(42)
uniform = np.random.uniform(0, 1, 10)

# 新接口
rng = np.random.default_rng(42)
uniform = rng.uniform(0, 1, 10)

```

两套 seed 互不影响

设置 `np.random.seed()` 不影响 `default_rng` 生成的序列, 反之亦然。

3.3 random vs np.random vs scipy.stats

模块	适用场景
random (Python 标准库)	简单需求, 每次生成 1 个样本
np.random	快速生成大规模数组 (向量化)
scipy.stats	额外提供 PDF、CDF、分位数等统计函数

第四章 数组拼接

4.1 一维数组拼接

```

a = np.array([1, 2, 3, 4])
b = np.array([5, 6, 7, 8])

np.concatenate([a, b])      # [1,2,3,4,5,6,7,8] — 基础拼接
np.vstack([a, b])           # [[1,2,3,4],[5,6,7,8]] — 按行堆叠 (变 2D)
np.hstack([a, b])           # [1,2,3,4,5,6,7,8] — 按列拼接

```

```
np.column_stack([a, b])      # [[1,5],[2,6],[3,7],[4,8]] — 变为2列矩阵
np.stack([a, b], axis=0)    # 同 vstack
np.stack([a, b], axis=1)    # 同 column_stack
```

4.2 二维数组拼接

```
A = np.arange(6).reshape(2,3)      # shape (2,3)
B = np.arange(6,12).reshape(2,3)   # shape (2,3)

np.concatenate([A, B], axis=0)      # 按行拼接, 结果 (4,3)
np.concatenate([A, B], axis=1)      # 按列拼接, 结果 (2,6)
np.vstack([A, B])                   # 同 axis=0, 结果 (4,3)
np.hstack([A, B])                   # 同 axis=1, 结果 (2,6)
np.block([A, B], [B, A]))           # 分块矩阵
```

4.3 简洁语法：np.c_[] 和 np.r_[]

这两个「索引魔法」对象让拼接代码更直观，**注意用方括号 [] 而非圆括号 ()**：

表达式	效果
np.c_[a, b]	按列拼接 (≈ column_stack)
np.r_[a, b]	按行/元素拼接 (≈ hstack for 1D)
np.c_[A, B]	2D 矩阵按列拼接
np.r_[A, B]	2D 矩阵按行拼接
np.r_[1:10:2]	利用切片语法创建序列 [1,3,5,7,9]
np.c_[1:6]	创建列向量 (2D, shape (5,1))
np.c_[1:5, np.random.rand(4,3)]	切片列与随机矩阵拼接

第五章 索引与数据选取

5.1 位置索引

```
arr = np.arange(15)      # [0,1,...,14]
arr[5]                   # 取第 5 个元素 → 5

arr2d = np.arange(15).reshape(5,3)
arr2d[0, 2]              # 第 0 行第 2 列 → 2
arr2d[0]                 # 整个第 0 行 → [0,1,2]
```

5.2 切片 (Slicing)

```
arr[5:8]                  # 取索引 5,6,7
arr2d[:, 1:3]             # 所有行, 第 1、2 列
```

```
arr2d[:2, :]          # 前 2 行, 所有列
arr2d[:2]              # 同上 (省略 [:,1])
```

❗ 切片返回「视图」而非副本

对切片的修改会影响原数组！如需独立副本，使用 `arr[5:8].copy()`。

```
# 检查是否为视图
arr_slice = arr[5:8]
print(arr_slice.base is arr)  # True → 视图

# 强制复制
arr_copy = arr[5:8].copy()
print(arr_copy.base is arr)   # False → 副本
```

5.3 布尔索引

```
arr[arr > 2]          # 选出大于 2 的元素
arr[(arr > 2) & (arr < 8)] # 组合条件 (用 & | ~, 不用 and or not)
arr[~(arr > 5)]        # 等价于 arr[arr <= 5]
```

实际应用 — 按第一列过滤样本：

```
X[X[:, 0] > 0]        # 选出第一列特征 > 0 的所有样本 (行)
```

5.4 整数数组索引

```
arr = np.arange(0, 20, 2)  # [0,2,4,6,8,10,12,14,16,18]
arr[[5, 1, 7]]             # 取索引 5,1,7 处的元素 → [10, 2, 14]
```

❗ 整数数组索引返回副本

与切片不同，整数数组索引始终返回数据副本，修改结果不影响原数组。

5.5 数据清洗的核心思维

大多数数据预处理 = 选择特征 + 过滤样本 + 替换值

```
# 选择特征 (取前两列)
X[:, :2]

# 过滤样本 (只保留第一特征 > 0 的行)
X[X[:, 0] > 0]

# 替换值 (将负值置零)
X[X < 0] = 0

np.where(condition) # 返回一个元组, 包含满足条件的元素的索引数组。
在 np.where() 返回的元组中, 每个元素表示一个维度的索引。
输出如: (array([0, 0, 1, 2, 2]), array([0, 2, 1, 0, 2]))
np.where(condition, x, y) # 如果 condition 为 True, 从 x 中取值; 如果为 False, 从 y 中取
```

第六章 向量化与广播

6.1 向量化思维

向量化是 NumPy 的核心编程范式：用数组操作取代显式循环，代码像数学公式一样直观，同时速度更快、更安全。

```
# Python 循环（慢、易错）
for i in range(A.shape[0]):
    for j in range(A.shape[1]):
        X[i, j] = A[i, j] - B[i, j]

# NumPy 向量化（快、简洁）
X = A - B

# z-score 中心标准化（一行搞定）【每列的均值调整为 0、标准差调整为 1】
(X - X.mean(axis=0)) / X.std(axis=0)
```

6.2 广播机制（Broadcasting）

广播决定了形状不同的数组之间如何运算。它允许 NumPy 「扩展」较小的数组来匹配较大数组的形状，且无需实际复制数据。

广播规则

规则：从尾部维度开始对齐，若两个维度相等，或其中一个为 1，则可以广播。若不满足则报错。【沿着缺失的维度和长度为 1 的维度补充】

```
# 规则 1：标量广播到整个数组
X = X / 10          # 10 广播到 X 的每个元素

# 规则 2：1D 数组广播到矩阵的每行
# X.shape = (100, 3), mean.shape = (3,)
X - X.mean(axis=0)  # mean 自动广播到 100 行

# 规则 3：列向量广播到矩阵
# X.shape = (4, 3), col.shape = (4, 1)
X - col             # col 广播到 3 列
```

广播维度对齐示意

形状组合	广播结果
(4, 3) 与 (3,)	尾部 3=3, 广播 → (4, 3)
(4, 3) 与 (4, 1)	1 维广播 → (4, 3)
(3, 4, 2) 【深度 3, 4 行 2 列】 与 (4, 2)	尾部对齐 4=4, 2=2, 前补 1 → (3, 4, 2)

(3,4,2) 与 (4,1)	尾部对齐, 1 广播 $\rightarrow$ (3,4,2)
(3,) 与 (4,)	不相容! 报错

实战：keepdims 参数

当需要让结果保持 2D 形状（以便参与后续广播），使用 **keepdims=True**：

```
row_sums = P.sum(axis=1, keepdims=True) # shape (n,1) 而非 (n,)
P = P / row_sums                        # 正确！广播成立

# 如果不用 keepdims=True:
row_sums = P.sum(axis=1) # shape (n,)，是行向量
P = P / row_sums        # 错误！除法按列广播而非按行
```

第七章 通用函数 (Universal Functions)

7.1 什么是 ufunc?

ufunc (Universal Function) 是 NumPy 提供的、与数组数据结构配套的函数集合，具备三个特点：**快速、向量化、逐元素运算**

7.2 一元函数 (Unary)

函数	说明
np.abs(x)	绝对值
np.sqrt(x)	平方根
np.exp(x)	e^x
np.log(x)	自然对数
np.log2(x) / np.log10(x)	以 2/10 为底的对数
np.sign(x)	符号函数 (-1, 0, 1)
np.ceil(x) / np.floor(x)	向上/向下取整
np.cos(x) / np.sin(x)	三角函数

7.3 二元函数 (Binary)

函数	说明
np.add(a, b) 即 $a + b$	加法
np.subtract(a, b) 即 $a - b$	减法
np.multiply(a, b) 即 $a * b$	逐元素乘（不是矩阵乘）
np.divide(a, b) 即 a / b	除法

<code>np.power(a, b)</code> 即 <code>a ** b</code>	幂运算
<code>np.maximum(a, b)</code>	逐元素取最大值
<code>np.minimum(a, b)</code>	逐元素取最小值
<code>np.greater(a, b)</code> 即 <code>a > b</code>	大于（返回布尔数组）

7.4 聚合统计函数（带 axis 参数）

函数	说明
<code>X.sum(axis=0)</code>	各列求和
<code>X.mean(axis=0)</code>	各列均值
<code>X.std(axis=0)</code>	各列标准差
<code>X.var(axis=0)</code>	各列方差
<code>X.min(axis=0)</code> / <code>X.max(axis=0)</code>	各列最小/最大值
<code>X.argmin(axis=0)</code> / <code>X.argmax(axis=0)</code>	各列最小/最大值的索引
<code>np.median(X, axis=0)</code>	各列中位数
<code>X.cumsum(axis=0)</code>	各列累积求和
<code>np.argsort(X, axis=1)</code>	各行排序后的索引

综合描述统计示例

```
mean    = X.mean(axis=0)
std     = X.std(axis=0)
min_    = X.min(axis=0)
max_    = X.max(axis=0)
median  = np.median(X, axis=0)

# 用户总观看数
total_views = X.sum(axis=1)      # 每行求和
print('平均观看视频数:', total_views.mean())
```

Softmax 函数（自定义 ufunc 的典型例子）

Softmax 将任意实数向量转化为概率分布，是深度学习中最常见的激活函数之一：

```
def softmax(x):
    e = np.exp(x)                # 逐元素取指数
    return e / e.sum(axis=1, keepdims=True) # 归一化（keepdims 保持 2D）

P = softmax(P)                  # 每行之和变为 1，得到概率分布
```

第八章 线性代数

8.1 乘法：* 与 @

```
a = np.arange(4)      # [0, 1, 2, 3]

a * a                 # 逐元素乘: [0, 1, 4, 9]
a @ a                 # 内积 (点积): 0+1+4+9 = 14

A = a.reshape(2,2)    # [[0,1],[2,3]]
A * A                 # 逐元素乘: [[0,1],[4,9]]
A @ A                 # 矩阵乘: [[2,3],[6,11]]
```

è * 与 @ 的区别

* 始终是逐元素乘 (Hadamard product) 。@ 是内积/矩阵乘法 (Matrix multiplication) 。千万不要混用！

8.2 向量的行列性质

NumPy 中 1D 数组 (向量) 没有行/列之分, 它是「双用途」的:

```
A = np.arange(6).reshape(3,2)  # shape (3,2)
b1 = np.arange(3)               # shape (3,)   → 当左乘时视作行向量
b2 = np.arange(2)               # shape (2,)   → 当右乘时视作列向量

b1 @ A      # 行向量 (1,3) @ (3,2) → (2,) 结果
A @ b2      # (3,2) @ 列向量 (2,1) → (3,) 结果
```

8.3 常用线性代数函数 (numpy.linalg)

函数	说明
<code>np.linalg.inv(A)</code>	求逆矩阵 A^{-1}
<code>np.linalg.solve(A, b)</code>	解线性方程组 $Ax = b$ (比 <code>inv</code> 更数值稳定)
<code>np.linalg.det(A)</code>	行列式
<code>np.linalg.eig(A)</code>	特征值与特征向量
<code>np.linalg.svd(A)</code>	奇异值分解
<code>np.linalg.qr(A)</code>	QR 分解
<code>A.T</code>	转置
<code>np.trace(A)</code>	迹 (主对角线之和)
<code>np.diag(A)</code>	取对角元素 (或从向量构造对角矩阵)
<code>np.linalg.norm(r_new - r, ord=1)</code>	计算这个差值数组的范数。默认情况下计算的是 L2 范数 (欧几里得距离), <code>ord=1</code> 则可变为 L1 距离。

示例：最小二乘线性回归

```

# 数据生成
X = np.random.randn(100, 3)
w = np.array([1., 0.5, 2.])
b = 0.5
y = X @ w + b    # 真实标签 (y = Xw + b)

# 添加截距列
X_aug = np.hstack((np.ones((100,1)), X))    # shape (100,4)

# 最小二乘解: beta = (X'X)^{-1} X'y
from numpy.linalg import inv, solve

# 方法 1: 直接求逆 (简单但数值稳定性较差)
beta = inv(X_aug.T @ X_aug) @ X_aug.T @ y

# 方法 2: solve (推荐, 更稳定) 解方程: (X'X)beta=X'y
beta = solve(X_aug.T @ X_aug, X_aug.T @ y)
b_hat, *w_hat = beta
# 【解包语法】通常线性回归的第一个系数是截距 (bias), 后面的是各个特征的权重 (weights)

```

第九章 综合案例：马尔可夫链模拟视频推荐

9.1 背景与模型

马尔可夫链 (Markov Chain) 是一种随机过程，具有「无记忆」性质：下一状态的概率分布只由当前状态决定，与历史无关。

本案例模拟用户在视频网站（类 TikTok）上的视频浏览行为：

- **状态 (states)**：视频类别（搞笑、科技、美食、音乐、结束）
- **转移概率矩阵 P**： $P[i][j]$ = 看完第 i 类视频后，下一个看第 j 类的概率
- **初始分布 p_0** ：用户第一个视频的类别分布
- **吸收态「结束」**：用户停止刷视频后进入，不再转移到其他状态

9.2 生成转移概率矩阵

```

import numpy as np

states = ['搞笑', '科技', '美食', '音乐']
p0 = np.array([0.4, 0.3, 0.2, 0.1])    # 初始分布

np.random.seed(100)
P = np.random.uniform(-0.5, 0.5, size=(4,4))
P = P + np.eye(4)    # 主对角线加 1, 使同类视频继续看的概率更高

```

```
# Softmax 归一化 → 每行变为概率分布
def softmax(x):
    e = np.exp(x)
    return e / e.sum(axis=1, keepdims=True)

P = softmax(P)
```

9.3 稀疏化（只推荐 Top-k 类别）

```
# 只保留每行概率最高的 k=2 个类别（更真实的推荐场景）
k = 2
P_sparse = np.zeros_like(P)

for i in range(P.shape[0]):
    idx = np.argsort(P[i])[-k:] # 取最大的 k 个索引
    P_sparse[i, idx] = P[i, idx]

P = P_sparse / P_sparse.sum(axis=1, keepdims=True) # 重新归一化
```

9.4 添加吸收态「结束」

```
# 方法：使用 np.c_[] 和 np.vstack
states.append('结束')

# 添加一行：从各状态转移到「结束」的概率（随机 5%~20%）
P = np.c_[P, np.random.uniform(0.1, 0.2, size=P.shape[0])]

# 添加吸收态自身的行：[0,0,0,0, 1]
P = np.vstack([P, np.r_[np.zeros(P.shape[0]), 1]])

# 重新归一化（加了一列后行和不再是 1）
P = P / P.sum(axis=1, keepdims=True)

print(P.sum(axis=1)) # 应全为 1.0
```

9.5 模拟用户浏览路径

```
def simulate(P, p0, max_steps=500):
    state = np.random.choice(len(P), p=p0) # 根据初始分布选第一个视频
    path = [state]
    for _ in range(max_steps):
        state = np.random.choice(len(P), p=P[state]) # 按当前状态的概率行转移
        path.append(state)
        if state == len(P) - 1: # 到达「结束」状态则停止
            break
    return path
```

```
paths = [simulate(P, p0) for _ in range(5000)] # 模拟 5000 个用户
```

9.6 统计观看次数

```
def count_states(path, n_states):
    counts = np.zeros(n_states, dtype=int)
    for s in path:
        if s < n_states: # 不计入吸收态
            counts[s] += 1
    return counts

# 生成观看矩阵: 5000 行 × 4 列
X = np.array([count_states(p, 4) for p in paths])

# 描述统计
print('各类均值:', X.mean(axis=0))
print('各类标准差:', X.std(axis=0))
print('平均总观看数:', X.sum(axis=1).mean())
```

9.7 理论计算：基本矩阵法

对于带吸收态的转移矩阵 P ，其基本矩阵（Fundamental Matrix）为 $\mathbf{N} = (\mathbf{I} - \mathbf{Q})^{-1}$ ，其中 $\mathbf{Q}$ 是瞬态（非吸收态）部分的子矩阵。

含义： $N[i][j]$ 表示从状态 i 出发，在被吸收前，期望访问状态 j 的次数。

```
Q = P[:4, :4] # 提取瞬态子矩阵
N = np.linalg.inv(np.eye(4) - Q) # 基本矩阵

# 理论各类视频播放次数期望
expected_views = p0[:4] @ N
print('理论播放量:', expected_views)
print('模拟平均值:', X.mean(axis=0))

# 理论总播放量
total_expected = p0[:4] @ N.sum(axis=1)
print('理论总播放量:', total_expected)
print('模拟总均值:', X.sum(axis=1).mean())

# import math
# print("95%置信区间下届:", mean - 1.96*std/math.sqrt(5000))
# print("95%置信区间上届:", mean + 1.96*std/math.sqrt(5000))
print("总播放次数的期望:", p0[:4] @ N.sum(axis=1))
print("模拟总播放次数均值:", mean.sum())
```

第十章 速查手册

10.1 数组创建

操作	说明
<code>np.array(list)</code>	从列表创建
<code>np.zeros(n)</code> / <code>np.ones(n)</code>	全零/全一
<code>np.eye(n)</code>	单位矩阵
<code>np.arange(start, stop, step)</code>	等差序列
<code>np.linspace(start, stop, n)</code>	等分 n 个点
<code>np.random.randn(m, n)</code>	标准正态随机矩阵
<code>np.random.uniform(a,b,size)</code>	均匀分布随机数
<code>arr.reshape(m, n)</code>	重塑形状
<code>arr.copy()</code>	强制复制

10.2 拼接速查

操作	说明
<code>np.vstack([A,B])</code>	垂直堆叠（按行）
<code>np.hstack([A,B])</code>	水平堆叠（按列）
<code>np.c_[A, B]</code>	按列拼接（简洁写法）
<code>np.r_[A, B]</code>	按行拼接（简洁写法）
<code>np.c_[P, col_vec]</code>	矩阵右侧添加一列
<code>np.vstack([P, row_vec])</code>	矩阵底部添加一行

10.3 线性代数速查

操作	说明
<code>A @ B</code>	矩阵乘法
<code>A * B</code>	逐元素乘法
<code>A.T</code>	转置
<code>np.linalg.inv(A)</code>	求逆
<code>np.linalg.solve(A, b)</code>	解方程组（推荐）
<code>np.linalg.eig(A)</code>	特征分解
<code>np.linalg.svd(A)</code>	奇异值分解
<code>p0 @ P</code>	初始分布乘转移矩阵（马尔可夫链一步）

10.4 常见陷阱提醒

- 切片是视图：`arr[5:8]` 不复制数据，修改它会影响原数组。需要独立数据用 `.copy()`

- *** vs @**: * 是逐元素乘, @ 是矩阵乘。切勿混淆!
- **广播方向**: $X - X.mean(axis=0)$ 正确; 但要将列向量广播需 `reshape(-1,1)`
- **布尔索引用 & | ~**: 不能用 `and/or/not`, 否则报错
- **keepdims=True**: 需要广播时, 聚合结果保持维度, 如 `sum(axis=1, keepdims=True)`
- **solve 比 inv 好**: 数值精度更高, 求 $Ax=b$ 的解用 `np.linalg.solve` 而非 `inv(A)@b`

—— End of Notes ——